MTD0: Preliminary Disassembly Findings

Technical research note - revision 0.1 | 2 November 2026

Scope. Initial analysis of the Netgear ReadyNAS SPARC NAND boot image, focusing on control flow, processor initialization, low-level I/O, and NAND page access. This is a working reverse-engineering document, not a verified hardware reference manual.

Evidence boundary. The offsets and observations below are consolidated from our prior disassembly discussions. The original binary, complete opcode listing, board schematics, and GPL header files were not supplied for direct re-verification in this report. Therefore register functions, GPIO assignments, and hardware identities are not asserted beyond the available observations.

1. Working conventions

Field	Convention
Offsets	Hexadecimal byte offsets relative to the start of the MTD0 image.
Instruction listing	Absolute offset + four raw opcode bytes + SPARC mnemonic + English comment.
Interpretation	Separate observed instruction behavior from inferred purpose.
Status labels	Observed = reported from listing; inferred = plausible interpretation; unknown = requires source or hardware verification.

2. Image and platform context

Item	Current working information
Processor family	SPARC (exact CPU and SoC variant not verified here).
Flash layout	MTD0 approximately 1 MiB; MTD1 approximately 63 MiB; 64 MiB NAND overall (working notes).
NAND geometry	512-byte page payload and 16 KiB erase block (working notes; validate against device IDs / driver).
Firmware material	RAIDiator SPARC 4.1.16 material, TFTP recovery files, and extracted readynas_gpl source tree available locally.

3. Early boot and processor state

Offset(s)	Observed behavior / preliminary interpretation
0x02A4	Write 0 to %tbr (trap base register). Observed instruction-level action; boot context remains to be confirmed.
0x02B4-0x02D4	Read an alternate address-space value (AIUP in working notes), mask/test it, and select a path through %g2.

Offset(s)	Observed behavior / preliminary interpretation
0x02F0	Path associated with %g2 = 1 appears to stall or loop. Label as an observed control-flow pattern, not a confirmed failure mode.
0x030C	Branch target associated with %g2 = 0; tentatively called cold-start path.
%g2 = 2	Uses an indirect jump through the word at 0x30007FFC, with a delay-slot clear; tentatively called warm-start path.
0x0410	Write 2 to %wim (window invalid mask).
0x0420-0x042C	References test-pattern values 0x12345678 and 0x87654321. Their role requires confirmation.
0x0464	Store through an alternate address space, annotated ASI_PHYS_USE_EC in working notes.
0x047C	Store 0x78000000 through an alternate address space, annotated ASI_AIUP in working notes.

Interpretation caution: cold/warm boot labels are hypotheses based on branch behavior. The meaning of the AIUP field and ASI names must be checked against the processor headers or architecture reference.

4. Memory-mapped register candidates

Address	Observed access / working hypothesis
0x20000094	Written with 0x6000 during initialization. Peripheral identity unknown.
0x200000D0	Written with 0x2AD2 in the 0x030C-0x03F4 initialization region.
0x200000D8	Data register or staging register candidate; written during the same sequence.
0x200000E0	Polled on bit 0; likely status/ready handshake, exact device unknown.
0x2000010C	Bit 10 reported set; previously annotated PADRE_PCI_FEATURE. Symbol and function require source cross-check.
0x30007FFC	Memory word used as an indirect jump target for one boot-selection path.
0x08000000	Used in a NAND page read/copy routine (0x225C region).
0x08010000	Used in NAND command/address sequence (0x2218 region).
0x08020000	Used in NAND command/address sequence (0x2218 region).

Important: The table contains candidate MMIO addresses, not confirmed GPIO pins. An address-to-pin mapping requires SoC documentation, board-level connections, and/or driver definitions.

5. NAND routines and data path

Offset	Working interpretation	Confidence
0x21AC	NAND address translation / possible bad-block remapping.	Hypothesis
0x2218	NAND command and address cycles using 0x08010000 and 0x08020000.	Inferred
0x225C	NAND page read from 0x08000000 and transfer of 512 bytes.	Inferred

The apparent sequence is: compute/translate a logical NAND location; issue command/address transactions; read a page; copy its payload to memory. The precise command codes, spare/OOB handling, ECC path, and bad-block policy remain open.

6. Other identified code regions

Offset(s)	Observation
0x037FC	Function prologue using save with a 208-byte stack frame.
0x0058C-0x005F8	Region containing rett %l2; trap-return context needs verification.
0x00624-0x00648	Signed-division routine using sdivcc and retl.
0x006E4-0x00708	String-comparison-like routine (tentative).
0x0071C-0x0076C	Hexadecimal digit parsing routine (tentative).
0x00774	Function prologue with save and a 104-byte stack frame; branch toward 0x0080C noted.

7. Header-file investigation: recommended next phase

The extracted GPL tree may contain architecture headers, platform register definitions, board support code, and driver mappings. Search for the literal addresses first, then trace nearby macros and driver use sites. Do not assume a macro proves the exact NAS board wiring.

High-value search targets

Register bases and offsets: 0x20000094, 0x200000D0, 0x200000D8, 0x200000E0, 0x2000010C, 0x08000000, 0x08010000, 0x08020000, 0x30007FFC. Symbol candidates: PADRE, PCI_FEATURE, GPIO, GPI, GPO, PINMUX, MTD, NAND, ECC, IRQ, INTR, ASI, AIUP, WIM, TBR.

Reproducible local searches

```
cd ~/Projects/asm-linux-sparc/NetgearDuo
rg -n -i '0x200000(d0|d8|e0|94)|0x2000010c|0x080[012]0000' sources/readynas_gpl
rg -n -i 'padre|gpio|pinmux|nand|ecc|irq|asi_aiup' sources/readynas_gpl --glob '*.[chS]'
find sources/readynas_gpl -type f \( -name '*.h' -o -name '*.c' \) | wc -l
```

If ripgrep (rg) is unavailable, use grep -RniE with equivalent patterns. Search source files (.c and .S) as well as headers: register definitions are useful only when tied to actual accesses.

Suggested evidence table for the next revision

Address / symbol	Source path + line	Access / bit field	Status
0x200000D0	To be extracted	Bootloader write 0x2AD2	Unverified
0x200000E0	To be extracted	Bootloader polls bit 0	Unverified
GPIO bank / pin	To be extracted	Driver use + board signal	Unknown

8. Open questions and validation checklist

- Identify the exact SoC/CPU and match its register map against the GPL sources or vendor documentation.
- Confirm MTD0 binary SHA-256, size, byte order, and disassembly base offset.
- Check the SPARC branch delay slots and alternate address space semantics at every cited offset.
- Identify NAND command values, OOB/ECC processing, and the true bad-block mapping strategy.
- Resolve the 0x200000xx peripheral block and verify bit definitions against source code.
- Map GPIO register banks to physical pins only when board-specific wiring evidence is available.
- Document each finding with opcode bytes, instruction text, source file/line, and confidence level.

9. Next deliverable

Revision 0.2 should add a sourced register map (address, register name, bit definitions, file/line reference), a separate GPIO inventory, and annotated bootloader listings around 0x02A4-0x047C and 0x21AC-0x225C. The extracted GPL files are the priority evidence source; any missing pinout details should remain explicitly unknown.

Prepared as a preliminary research record for the agguro NetgearDuo project. No binary or header file was directly inspected in producing this revision.